

Introduction to SQL & Basic Querying

Day 1 practical workbook · MySQL 8.0 · Sales order dataset

This workbook turns one spreadsheet of 4,000 sales order lines into a working MySQL database, one statement at a time. Each part introduces a statement, runs it against the dataset and shows the result you should see. The statements were run on MySQL 8.0, and the results printed here are the real output.

Part	Topic	What you do
1	Creating a database	Create and select <code>sales_db</code>
2	Creating a table	Build <code>sales_raw</code> , a table shaped like the spreadsheet
3	Data types	See why each of the 21 columns got its type
4	Data models, ER diagrams and schema design	Split the flat sheet into four related tables
5	Inserting data	Insert rows by hand, then load all 4,000 rows
6	Creating a user	Add three accounts
7	Granting permissions	Give each account only what it needs
8	Data retrieval	<code>SELECT</code> , <code>DISTINCT</code> , <code>WHERE</code> , <code>ORDER BY</code> and <code>LIMIT</code>
9	Aliases	Rename columns and calculations in a result
10	Comments	Document a query and switch lines off while testing
11	Temporary tables	Keep an intermediate result for your session
12	CTEs	Name a sub-result inside a single query
A, B	Appendices	Common errors and what they mean; answers to the exercises

Before you start

- **Software:** MySQL Server 8.0 and MySQL Workbench, connected as `root` (the administrator account created during installation).
- **Data:** `sales_orders.csv`, the dataset saved as CSV. Save it in a folder with a short path, such as `C:/sql_class/`. Part 5 explains how it was made from `sales_order_dataset.xlsx`.
- **Optional:** `sql_day1_class_script.sql` holds every statement in this workbook in order. `sales_raw_inserts.sql` is a fallback for loading the data if the import gives trouble.

Running statements in MySQL Workbench

- Type statements in the query tab. Every statement ends with a semicolon.
- **Ctrl+Enter** runs the statement the cursor is in. **Ctrl+Shift+Enter** runs everything in the tab, or only the selected text if you have selected some. On a Mac use `Cmd` in place of `Ctrl`.
- Query results appear in the grid under the editor. Messages such as `730 row(s) affected` and all errors appear in the Output panel at the bottom.

- SQL keywords are not case sensitive. This workbook writes keywords in CAPITALS and our own names in lowercase so the two are easy to tell apart.

The dataset

Each row of the spreadsheet is one product line on an order: 4,000 rows and 21 columns. The columns describe four different things, which matters in Part 4:

About	Spreadsheet columns
The order	Order ID, Order Date, Ship Date, Ship Mode
The customer	Customer ID, Customer Name, Segment
Where it was shipped	Country, City, State, Postal Code, Region
The product	Product ID, Category, Sub-Category, Product Name
The numbers for that line	Sales, Quantity, Discount, Profit, SalesCategory

Part 1 Creating a database

A database is a named container for tables that belong together. MySQL can hold many databases on one server, so the first job is to create one for this project.

```
CREATE DATABASE sales_db;
```

OUTPUT 1 row(s) affected

List the databases on the server to confirm it exists. The other four are MySQL's own system databases; leave them alone.

```
SHOW DATABASES;
```

RESULT

Database
information_schema
mysql
performance_schema
sales_db
sys

Watch out: Creating a database does not select it. Until you say which database you are working in, statements fail with Error Code: 1046. No database selected.

```
USE sales_db;
SELECT DATABASE();
```

RESULT

DATABASE()
sales_db

USE lasts until you disconnect, so run it again each time you reconnect. In Workbench the active database is shown in bold in the Schemas panel on the left.

Running it twice

Run `CREATE DATABASE sales_db;` a second time and MySQL refuses:

```
CREATE DATABASE sales_db;
```

ERROR

```
Error Code: 1007. Can't create database 'sales_db'; database exists
```

That is an error, but a harmless one. If you want a script that can be re-run without stopping, add `IF NOT EXISTS`:

```
CREATE DATABASE IF NOT EXISTS sales_db;
```

Watch out: `DROP DATABASE sales_db;` deletes the database and every table in it, with no confirmation and no undo. It is useful for starting an exercise again and dangerous everywhere else.

TRY IT YOURSELF

Create a second database called `practice_db`, check that it appears in `SHOW DATABASES`, then drop it. Finish with `USE sales_db;` so you are back in the right place.

Part 2 Creating a table

A table has a name and a list of columns. Each column has a name, a data type and, optionally, constraints (rules the data must obey).

```
CREATE TABLE table_name (
    column_name data_type constraint,
    column_name data_type constraint,
    ...
);
```

- Columns are separated by commas. There is no comma after the last one.
- Names should be lowercase with underscores: no spaces, no hyphens. The spreadsheet heading `Order ID` becomes `order_id` and `Sub-Category` becomes `sub_category`.

Our first table: a copy of the spreadsheet

The first table mirrors the spreadsheet exactly, one column per spreadsheet column. Call it `sales_raw` because it will hold the data as it arrived, before any tidying. Part 3 explains the type chosen for each column.

```
CREATE TABLE sales_raw (
  order_id      VARCHAR(20),
  order_date    DATE,
  ship_date     DATE,
  ship_mode     VARCHAR(20),
  customer_id   VARCHAR(10),
  customer_name VARCHAR(100),
  segment       VARCHAR(20),
  country       VARCHAR(50),
  city          VARCHAR(50),
  state         VARCHAR(50),
  postal_code   VARCHAR(10),
  region        VARCHAR(20),
  product_id    VARCHAR(20),
  category      VARCHAR(30),
  sub_category  VARCHAR(30),
  product_name  VARCHAR(255),
  sales         DECIMAL(10,4),
  quantity      INT,
  discount      DECIMAL(4,2),
  profit        DECIMAL(10,4),
  sales_category VARCHAR(10)
);
```

OUTPUT 0 row(s) affected

A new table reports 0 row(s) affected because it is empty. Check that it exists and look at its structure:

```
SHOW TABLES;
```

RESULT

```
Tables_in_sales_db
sales_raw
```

```
DESCRIBE sales_raw;
```

RESULT

Field	Type	Null	Key	Default	Extra
order_id	varchar(20)	YES		NULL	
order_date	date	YES		NULL	
ship_date	date	YES		NULL	
ship_mode	varchar(20)	YES		NULL	
customer_id	varchar(10)	YES		NULL	
customer_name	varchar(100)	YES		NULL	
segment	varchar(20)	YES		NULL	
country	varchar(50)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postal_code	varchar(10)	YES		NULL	

Field	Type	Null	Key	Default	Extra
region	varchar(20)	YES		NULL	
product_id	varchar(20)	YES		NULL	
category	varchar(30)	YES		NULL	
sub_category	varchar(30)	YES		NULL	
product_name	varchar(255)	YES		NULL	
sales	decimal(10,4)	YES		NULL	
quantity	int	YES		NULL	
discount	decimal(4,2)	YES		NULL	
profit	decimal(10,4)	YES		NULL	
sales_category	varchar(10)	YES		NULL	

Null = YES means the column may be left empty, and the Key column is blank because this table has no keys yet. That is deliberate. `sales_raw` is only a landing area; the tables in Part 4 get keys and rules.

Fixing a mistake

If you mistyped a column, the simplest fix on an empty table is to drop it and create it again. `DROP TABLE` deletes the table and all its rows.

```
DROP TABLE IF EXISTS sales_raw;
```

TRY IT YOURSELF

Create a table called `regions` with two columns: `region_name VARCHAR(20)` and `manager VARCHAR(50)`. Check it with `DESCRIBE`, then drop it.

Part 3 Data types

Every column has exactly one data type. The type decides what the column will accept, how much space it uses and what you can do with it: you can add up numbers and subtract dates, but not if they were stored as text.

The types used in this dataset

Type	Stores	Used for	Example value
VARCHAR(n)	Text of varying length, up to n characters	IDs, names, cities, categories	'Claire Gute'
CHAR(n)	Text of exactly n characters	Postal code in the orders table (Part 4)	'01040'
INT	Whole numbers	quantity	3
DECIMAL(p, s)	Exact numbers: p digits in total, s of them after the decimal point	sales, profit, discount	957.5775
DATE	A calendar date: year, month, day	order_date, ship_date	'2016-11-08'

Other types you will meet later: `TEXT` for long passages such as reviews, `FLOAT` and `DOUBLE` for approximate scientific values, `TIME` for a time of day, and `DATETIME` for a date and time together, such as a login timestamp.

Five decisions behind the `sales_raw` table

- 1. An ID that contains letters is text.** `CA-2016-152156` and `FUR-B0-10001798` cannot be numbers. Even an ID made only of digits is usually better as text, because you will never add IDs together.
- 2. A postal code is text, not a number.** Excel treated Postal Code as a number and dropped the leading zero, so Holyoke, Massachusetts appears in the spreadsheet as `1040` where the real code is `01040`. This affects 160 of the 4,000 rows. Text keeps the zero. The same applies to cellphone numbers such as `0821234567`.
- 3. Money is DECIMAL, not FLOAT.** `FLOAT` and `DOUBLE` store approximations, and the errors show up when you add values. In the query below, `0.1E0` is a floating-point value and plain `0.1` is an exact decimal:

```
SELECT 0.1E0 + 0.2E0 AS float_sum,
       0.1   + 0.2   AS decimal_sum;
```

RESULT

float_sum	decimal_sum
0.300000000000000004	0.3

`DECIMAL(10,4)` allows 10 digits with 4 after the decimal point, so values up to 999999.9999. The largest sale in the dataset is 22638.48, and some values carry four decimals (957.5775), so nothing is rounded on the way in. `discount` is always between 0.00 and 0.80, so `DECIMAL(4,2)` is enough.

- 4. A date is a DATE, not text.** MySQL writes dates as `'YYYY-MM-DD'`, in quotes. Stored as a real date, a value can be compared, sorted and used in date arithmetic:

```
SELECT DATEDIFF('2016-11-11', '2016-11-08') AS days_between;
```

RESULT

days_between
3

- 5. VARCHAR needs room for the longest value.** The longest product name in the dataset is 127 characters, so `product_name` is `VARCHAR(255)`. A value longer than the limit is rejected with `Error Code: 1406. Data too long for column`, so leave headroom. `VARCHAR` only uses the space each value needs, so a generous limit costs nothing.

TRY IT YOURSELF

Which type would you give each of these columns, and why? (a) a customer's cellphone number, (b) a product's weight in kilograms, (c) a customer's date of birth, (d) the number of items in stock.

Part 4 Data models, ER diagrams and designing the schema

`sales_raw` has the same shape as the spreadsheet, and the same weakness. Look at rows 2 and 3 of the spreadsheet: one order with two products. The order date, the customer's name and the full address are typed out twice, and only the product columns differ.

- **Repetition.** A customer's name is stored once for every line they have ever ordered.

- **Updates go wrong.** If a customer changes their name, every one of those rows must be changed. Miss one and the database disagrees with itself.
- **It has already happened here.** 27 product IDs in the dataset appear with two different product names. In one flat table nothing prevents that.

The fix is to store each fact once, in a table about the thing that fact describes. Deciding which tables to have, what goes in each and how they connect is called **data modelling**, and the result is the database **schema**.

Step 1: find the entities

An entity is a thing the business keeps facts about. For each column ask: what is this a fact about? In this dataset there are four answers, so there will be four tables.

Table	One row is	Columns	Rows
customers	one customer	customer_id, customer_name, segment	730
products	one product	product_id, product_name, category, sub_category	1,615
orders	one order	order_id, order_date, ship_date, ship_mode, customer_id, country, city, state, postal_code, region	1,980
order_items	one product line on one order	order_item_id, order_id, product_id, sales, quantity, discount, profit	4,000

Two placements need a second look:

- **City, state, postal code and region go in orders, not customers.** Test it against the data: does a customer always have the same city? No. Sean O'Donnell has orders shipped to Fort Lauderdale, Florida and to Hempstead, New York, and 564 of the 730 customers ship to more than one city. The address is a fact about the order: where it was sent.
- **SalesCategory is left out.** It is calculated from Sales (High, Medium or Low). Storing a value that can be calculated means two things that can disagree. Calculate it in a query when it is needed.

Step 2: give every table a primary key

A **primary key** is the column that identifies one row. Its values must be unique and can never be empty. Three tables already have a natural one: `customer_id`, `product_id` and `order_id`.

`order_items` does not. The pair (`order_id`, `product_id`) looks unique, but five orders in the dataset list the same product on two separate lines. So we add our own key, `order_item_id`, and let MySQL number the rows with `AUTO_INCREMENT`.

Step 3: connect the tables with foreign keys

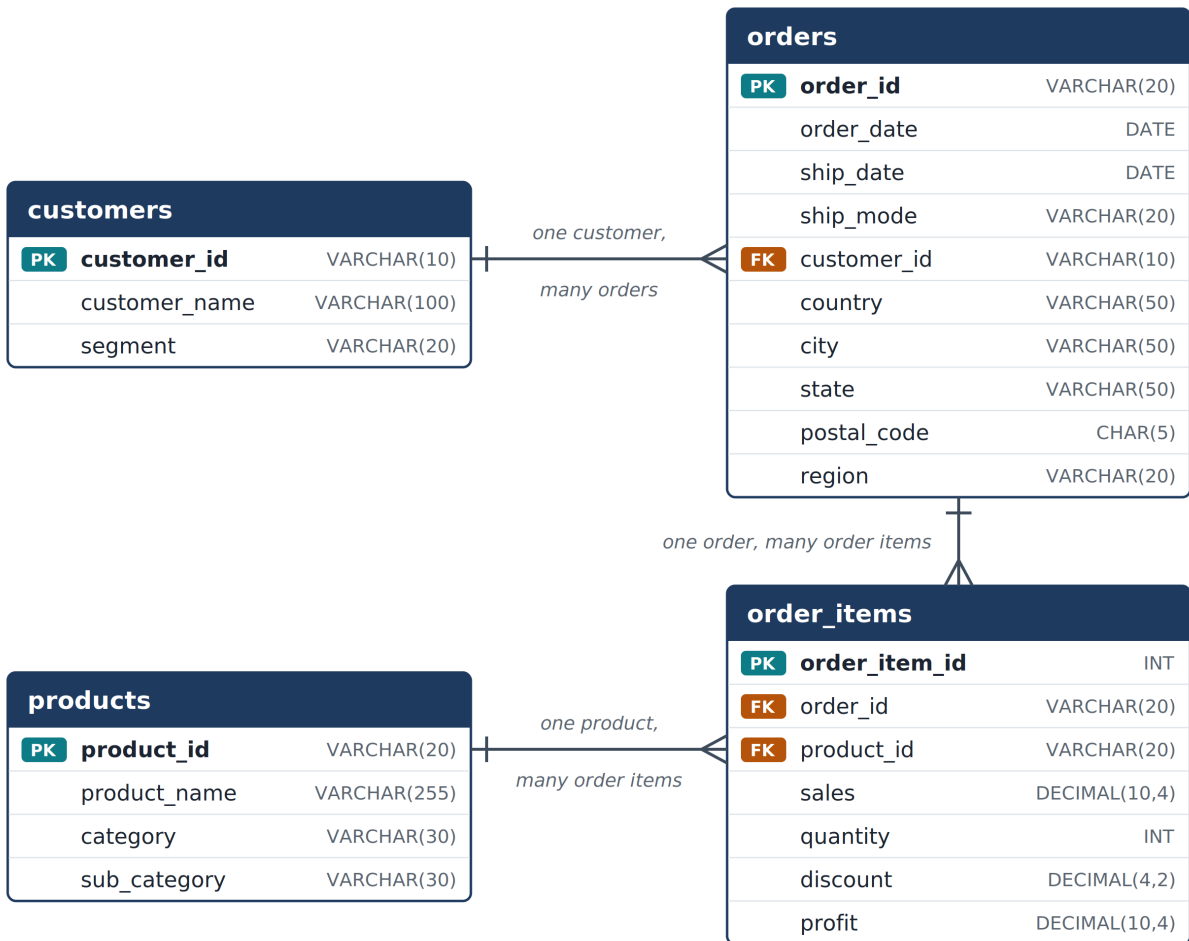
A **foreign key** is a column that holds the primary key of a row in another table. `orders.customer_id` says which customer placed the order. MySQL enforces it: an order cannot point to a customer that does not exist.

- One customer places many orders: `orders.customer_id` refers to `customers.customer_id`.
- One order has many order items: `order_items.order_id` refers to `orders.order_id`.
- One product appears on many order items: `order_items.product_id` refers to `products.product_id`.

All three are **one-to-many** relationships. The table on the "many" side holds the foreign key.

Step 4: draw it as an ER diagram

An entity-relationship (ER) diagram is the picture of a data model. Each box is a table. A line joins a primary key to the foreign key that refers to it: the end with a single bar is the "one" side and the end that fans out into three (a crow's foot) is the "many" side.



Tip: Workbench can draw this diagram from a real database. Once the tables exist, choose Database > Reverse Engineer (Ctrl+R) and follow the wizard.

Step 5: create the tables

Order matters. A table must exist before another table can refer to it, so create the two parent tables first, then **orders**, then **order_items**.

```
CREATE TABLE customers (
  customer_id  VARCHAR(10) PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  segment     VARCHAR(20)  NOT NULL
);
```

```
CREATE TABLE products (
  product_id    VARCHAR(20)    PRIMARY KEY,
  product_name  VARCHAR(255)   NOT NULL,
  category      VARCHAR(30)    NOT NULL,
  sub_category  VARCHAR(30)    NOT NULL
);
```

```
CREATE TABLE orders (
  order_id      VARCHAR(20)    PRIMARY KEY,
  order_date    DATE           NOT NULL,
  ship_date     DATE,
  ship_mode     VARCHAR(20),
  customer_id   VARCHAR(10)    NOT NULL,
  country       VARCHAR(50),
  city          VARCHAR(50),
  state         VARCHAR(50),
  postal_code    CHAR(5),
  region        VARCHAR(20),
  FOREIGN KEY (customer_id) REFERENCES customers (customer_id)
);
```

```
CREATE TABLE order_items (
  order_item_id INT            AUTO_INCREMENT PRIMARY KEY,
  order_id      VARCHAR(20)    NOT NULL,
  product_id    VARCHAR(20)    NOT NULL,
  sales         DECIMAL(10,4)  NOT NULL,
  quantity      INT            NOT NULL,
  discount      DECIMAL(4,2)   NOT NULL DEFAULT 0,
  profit        DECIMAL(10,4),
  FOREIGN KEY (order_id) REFERENCES orders (order_id),
  FOREIGN KEY (product_id) REFERENCES products (product_id)
);
```

Constraint	What it enforces
PRIMARY KEY	Identifies the row. Values must be unique and cannot be empty.
NOT NULL	The column must always be given a value. <code>ship_date</code> is left without it because an order that has not shipped yet has no ship date.
FOREIGN KEY ... REFERENCES	The value must already exist as a primary key in the other table.
AUTO_INCREMENT	MySQL supplies the next number (1, 2, 3 ...) when a row is inserted.
DEFAULT 0	The value used when an insert does not supply one.

Watch out: Write foreign keys on their own line, as above. MySQL also accepts the short form `customer_id VARCHAR(10) REFERENCES customers (customer_id)` on the column itself, but it silently ignores it: no error, and no foreign key is created.

Check the result. PRI marks a primary key and MUL marks a column that is indexed because it is a foreign key.

```
SHOW TABLES;
```

RESULT

```
Tables_in_sales_db
```

```
customers
```

```
order_items
```

```
orders
```

```
products
```

```
sales_raw
```

```
DESCRIBE order_items;
```

RESULT

Field	Type	Null	Key	Default	Extra
order_item_id	int	NO	PRI	NULL	auto_increment
order_id	varchar(20)	NO	MUL	NULL	
product_id	varchar(20)	NO	MUL	NULL	
sales	decimal(10,4)	NO		NULL	
quantity	int	NO		NULL	
discount	decimal(4,2)	NO		0.00	
profit	decimal(10,4)	YES		NULL	

TRY IT YOURSELF

The business wants to record returns: which order item was returned, on what date, and the reason. Design a returns table. Decide the columns and types, the primary key and the foreign key, then write the `CREATE TABLE` statement.

Part 5 Inserting data

`INSERT INTO` adds rows to a table. Name the table, list the columns, then give one value per column in the same order.

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

One row

Take the first customer in the spreadsheet. Text and dates go in single quotes; numbers do not.

```
INSERT INTO customers (customer_id, customer_name, segment)
VALUES ('CG-12520', 'Claire Gute', 'Consumer');
```

OUTPUT 1 row(s) affected

Several rows in one statement

Separate the rows with commas. The third customer is Sean O'Donnell, whose name contains an apostrophe. MySQL would read that apostrophe as the end of the text, so write it twice.

```
INSERT INTO customers (customer_id, customer_name, segment)
VALUES
  ('DV-13045', 'Darrin Van Huff', 'Corporate'),
  ('SO-20335', 'Sean O''Donnell', 'Consumer');
```

OUTPUT 2 row(s) affected

```
SELECT * FROM customers;
```

RESULT

customer_id	customer_name	segment
CG-12520	Claire Gute	Consumer
DV-13045	Darrin Van Huff	Corporate
SO-20335	Sean O'Donnell	Consumer

A row that refers to another table

Now the first order. Its `customer_id` must match a customer that already exists, which is why the customer went in first.

```
INSERT INTO orders (order_id, order_date, ship_date, ship_mode, customer_id,
  country, city, state, postal_code, region)
VALUES ('CA-2016-152156', '2016-11-08', '2016-11-11', 'Second Class', 'CG-12520',
  'United States', 'Henderson', 'Kentucky', '42420', 'South');
```

OUTPUT 1 row(s) affected

What the database refuses

The constraints from Part 4 now do their job. Insert Claire Gute a second time:

```
INSERT INTO customers (customer_id, customer_name, segment)
VALUES ('CG-12520', 'Claire Gute', 'Consumer');
```

ERROR

Error Code: 1062. Duplicate entry 'CG-12520' for key 'customers.PRIMARY'

The primary key stopped a duplicate. Now try an order for a customer who does not exist:

```
INSERT INTO orders (order_id, order_date, customer_id)
VALUES ('ZA-2026-000001', '2026-10-05', 'XX-99999');
```

ERROR

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails
 (`sales\_db`.`orders`, CONSTRAINT `orders\_ibfk\_1` FOREIGN KEY (`customer\_id`) REFERENCES `customers` (`customer\_id`))

The foreign key stopped an order with no customer. Neither of these could be prevented in a spreadsheet.

Removing the practice rows

Before loading the full dataset, remove the rows typed by hand so they are not loaded twice. Delete the order first: MySQL will not delete a customer who still has orders.

```
DELETE FROM orders
WHERE order_id = 'CA-2016-152156';

DELETE FROM customers
WHERE customer_id IN ('CG-12520', 'DV-13045', 'SO-20335');
```

OUTPUT 1 row(s) affected | 3 row(s) affected

Watch out: Always give DELETE a WHERE. Without one it removes every row in the table. Workbench guards against this with safe update mode, which rejects a DELETE whose WHERE does not use a key column (Error Code: 1175).

Loading the whole dataset

Nobody types 4,000 rows. The data travels in three steps:

Step	From	To	How
1	sales_order_dataset.xlsx	sales_orders.csv	Save the sheet as CSV
2	sales_orders.csv	sales_raw	Import the file into the landing table
3	sales_raw	customers, products, orders, order_items	INSERT ... SELECT

Step 1: from Excel to CSV

MySQL cannot read an .xlsx file. It reads plain text files where the values are separated by commas (CSV). The supplied sales_orders.csv is ready to use. To make it yourself from the spreadsheet:

1. Select the Order Date and Ship Date columns, open Format Cells, choose Custom and type yyyy-mm-dd. MySQL reads dates as year, month, day, and rejects a date written month-first such as 11/8/2016.
2. Choose File > Save As and pick **CSV UTF-8 (Comma delimited)**. UTF-8 keeps names such as Resi Pölking and Résumé Paper intact.

Watch out: Do not open sales_orders.csv in Excel and save it again. Excel rewrites the dates in your computer's regional format on the way out.

Step 2: import the CSV into sales_raw

There are two ways. Use one of them, not both, or every row is loaded twice.

Option A: the Table Data Import Wizard. No set-up is needed. It inserts one row at a time, so 4,000 rows take a minute or two.

1. In the Schemas panel, expand sales_db and then Tables. Right-click sales_raw and choose Table Data Import Wizard.
2. Browse to sales_orders.csv and click Next.
3. Choose **Use existing table** and select sales_db.sales_raw. Click Next.

4. Check that the encoding is utf-8 and that each source column is matched to the destination column of the same name. Click Next.
5. Click Next to run the import. When it finishes, the wizard reports how many records were imported: it should be 4,000.

Option B: LOAD DATA. One statement, and it finishes in under a second. For safety, loading a file from your own computer is switched off by default and must be allowed in two places first:

1. On the server, run the `SET GLOBAL` statement below. It stays on until the server restarts.
2. In Workbench, go to the home screen, right-click your connection and choose Edit Connection. On the Advanced tab, add the line `OPT_LOCAL_INFILE=1` to the Others box. Close the dialog and reconnect.

```
SET GLOBAL local_infile = 1;
```

```
LOAD DATA LOCAL INFILE 'C:/sql_class/sales_orders.csv'
INTO TABLE sales_raw
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

OUTPUT 4000 row(s) affected

Line	Meaning
<code>LOAD DATA LOCAL INFILE '...'</code>	Read a file from your own computer. Change the path to where you saved the file, and use forward slashes even on Windows.
<code>INTO TABLE sales_raw</code>	The table that receives the rows. The file's columns must be in the same order as the table's.
<code>CHARACTER SET utf8mb4</code>	The file is UTF-8 text.
<code>FIELDS TERMINATED BY ','</code>	Values are separated by commas.
<code>OPTIONALLY ENCLOSED BY '"'</code>	A value may be wrapped in double quotes. Product names that contain a comma are.
<code>LINES TERMINATED BY '\r\n'</code>	Each row ends with a Windows line ending, which is what Excel writes.
<code>IGNORE 1 ROWS</code>	Skip the heading row.

If `LOAD DATA` fails with Error Code: 3948 or Error Code: 2068, one of the two set-up steps above has not taken effect. If neither option works on your machine, open `sales_raw_inserts.sql` in Workbench and run it: it fills `sales_raw` with ordinary `INSERT` statements.

Check the import

Count the rows before going any further.

```
SELECT COUNT(*) AS rows_loaded
FROM sales_raw;
```

RESULT

rows_loaded

4000

```
SELECT order_id, order_date, customer_name, product_id, sales
FROM sales_raw
LIMIT 5;
```

RESULT

order_id	order_date	customer_name	product_id	sales
CA-2016-152156	2016-11-08	Claire Gute	FUR-BO-10001798	261.9600
CA-2016-152156	2016-11-08	Claire Gute	FUR-CH-10000454	731.9400
CA-2016-138688	2016-06-12	Darrin Van Huff	OFF-LA-10000240	14.6200
US-2015-108966	2015-10-11	Sean O'Donnell	FUR-TA-10000577	957.5775
US-2015-108966	2015-10-11	Sean O'Donnell	OFF-ST-10000760	22.3680

Tip: If the count is wrong (0, or 8,000 because the import ran twice), empty the table with `TRUNCATE TABLE sales_raw;` and import again.

Watch out: Workbench quietly adds `LIMIT 1000` to queries, so `SELECT * FROM sales_raw;` shows 1,000 rows even though 4,000 are there. Use `COUNT(*)` to count rows; do not read the count off the grid.

Step 3: fill the four tables from sales_raw

`INSERT` can take its rows from a query in place of a `VALUES` list. Whatever the `SELECT` returns is inserted. Load the parent tables first, for the same reason they were created first.

Customers. Each customer appears in `sales_raw` once per line they ordered. `DISTINCT` removes the repeats and leaves one row per customer.

```
INSERT INTO customers (customer_id, customer_name, segment)
SELECT DISTINCT customer_id, customer_name, segment
FROM sales_raw;
```

OUTPUT 730 row(s) affected

Products. Try the same approach:

```
INSERT INTO products (product_id, product_name, category, sub_category)
SELECT DISTINCT product_id, product_name, category, sub_category
FROM sales_raw;
```

ERROR

Error Code: 1062. Duplicate entry 'FUR-CH-10001146' for key 'products.PRIMARY'

The primary key has caught a real problem in the data. Look at that product ID:

```
SELECT DISTINCT product_id, product_name
FROM sales_raw
WHERE product_id = 'FUR-CH-10001146';
```

RESULT

product_id	product_name
FUR-CH-10001146	Global Value Mid-Back Manager's Chair, Gray
FUR-CH-10001146	Global Task Chair, Black

One ID, two names, so `DISTINCT` returns two rows and the second one breaks the primary key. This is one of the 27 product IDs mentioned in Part 4. Nothing was inserted: when a statement fails, MySQL undoes all of it.

In a real project you would ask the owner of the data which name is correct. For this exercise, keep one name per product ID. `GROUP BY product_id` produces one row per ID, and `MIN(product_name)` picks the name that comes first alphabetically. `GROUP BY` is covered properly on Day 2.

```
INSERT INTO products (product_id, product_name, category, sub_category)
SELECT product_id, MIN(product_name), category, sub_category
FROM sales_raw
GROUP BY product_id, category, sub_category;
```

OUTPUT 1615 row(s) affected

Orders. One row per order, so `DISTINCT` again. This is also where the postal codes are repaired:

`LPAD(postal_code, 5, '0')` pads the value on the left with zeros until it is five characters long, turning `1040` back into `01040`.

```
INSERT INTO orders (order_id, order_date, ship_date, ship_mode, customer_id,
                    country, city, state, postal_code, region)
SELECT DISTINCT order_id, order_date, ship_date, ship_mode, customer_id,
                country, city, state, LPAD(postal_code, 5, '0'), region
FROM sales_raw;
```

OUTPUT 1980 row(s) affected

Order items. Every row of `sales_raw` is one order item, so no `DISTINCT` is needed. `order_item_id` is not in the column list: `AUTO_INCREMENT` fills it in.

```
INSERT INTO order_items (order_id, product_id, sales, quantity, discount, profit)
SELECT order_id, product_id, sales, quantity, discount, profit
FROM sales_raw;
```

OUTPUT 4000 row(s) affected

Step 4: verify the load

Count each table and compare with the design in Part 4.

```
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM order_items;
```

Table	Expected rows
customers	730
products	1,615
orders	1,980
order_items	4,000

Row counts can match while values are wrong, so also check a total. Sales added up in the new table must equal sales added up in the raw table.

```
SELECT SUM(sales) AS total_sales FROM sales_raw;
SELECT SUM(sales) AS total_sales FROM order_items;
```

RESULT

```
total_sales
923303.1530
```

RESULT

total_sales

923303.1530

Last, confirm the postal code repair:

```
SELECT DISTINCT city, state, postal_code
FROM orders
WHERE state = 'Massachusetts'
ORDER BY city
LIMIT 5;
```

RESULT

city	state	postal_code
Everett	Massachusetts	02149
Franklin	Massachusetts	02038
Holyoke	Massachusetts	01040
Lawrence	Massachusetts	01841
Leominster	Massachusetts	01453

The spreadsheet is now a relational database. `sales_raw` has done its job. Keep it for now as a reference; in a real project it would be dropped or emptied once the load is verified.

TRY IT YOURSELF

Insert yourself as a customer: use the ID `ZZ-00001`, your own name and the segment `Consumer`. Confirm the row is there with a `SELECT`, then delete it.

Part 6 Creating a user

So far everything has been done as `root`, which can do anything on the server, including dropping every database. Real systems give each person or application its own account that can do only what the job requires. That takes two steps: create the user (this part), then grant permissions (Part 7).

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

- `username` is the name the person logs in with.
- `host` is where they may connect from. `localhost` means only from the machine MySQL runs on. `%` means from anywhere, which is less secure.
- `password` is their password. The ones below are for class only.

Create three users, one for each kind of job:

```
CREATE USER 'sales_analyst'@'localhost' IDENTIFIED BY 'Analyst#2026';
CREATE USER 'order_clerk'@'localhost' IDENTIFIED BY 'Clerk#2026';
CREATE USER 'sales_admin'@'localhost' IDENTIFIED BY 'Admin#2026';
```

MySQL keeps its users in a system table. `root` and the three accounts whose names start with `mysql.` were created by MySQL itself.

```
SELECT user, host
FROM mysql.user;
```

RESULT

user	host
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
order_clerk	localhost
root	localhost
sales_admin	localhost
sales_analyst	localhost

A new user can log in, and nothing else

To try an account in Workbench, go to the home screen, click the + next to MySQL Connections, enter a connection name and the username `sales_analyst`, then open it and type the password. Run these as `sales_analyst`:

RUN AS SALES_ANALYST

```
SHOW DATABASES;
```

RESULT

Database
information_schema
performance_schema

RUN AS SALES_ANALYST

```
USE sales_db;
```

ERROR

```
Error Code: 1044. Access denied for user 'sales_analyst'@'localhost' to database 'sales_db'
```

`sales_db` is not even visible. A new user has no permissions until some are granted.

Changing a password and removing a user

These two are for reference. Do not run them now: Part 7 needs `sales_analyst` with its original password.

```
ALTER USER 'sales_analyst'@'localhost' IDENTIFIED BY 'NewPassword#2026';
```

```
DROP USER 'sales_analyst'@'localhost';
```

Part 7 Granting permissions

GRANT gives a user permission to run particular statements on particular tables. Go back to the root connection for this part.

```
GRANT permission_list ON database_name.table_name TO 'username'@'host';
```

ON ...	The permission applies to
sales_db.orders	one table
sales_db.*	every table in one database
.	every database on the server

Permission	Allows
SELECT	Reading rows
INSERT	Adding rows
UPDATE	Changing existing rows
DELETE	Removing rows
CREATE, ALTER, DROP	Creating, changing and deleting tables
ALL PRIVILEGES	Everything at that level

A read-only analyst

The analyst writes reports. They need to read every table and change nothing.

```
GRANT SELECT ON sales_db.* TO 'sales_analyst'@'localhost';
```

A clerk who captures orders

The clerk looks things up in every table, but adds and corrects rows only in `orders` and `order_items`. That takes a database-level grant plus two table-level grants.

```
GRANT SELECT ON sales_db.* TO 'order_clerk'@'localhost';
GRANT INSERT, UPDATE ON sales_db.orders TO 'order_clerk'@'localhost';
GRANT INSERT, UPDATE ON sales_db.order_items TO 'order_clerk'@'localhost';
```

An administrator for this database

The administrator can do anything inside `sales_db`, and nothing outside it.

```
GRANT ALL PRIVILEGES ON sales_db.* TO 'sales_admin'@'localhost';
```

Checking what a user can do

```
SHOW GRANTS FOR 'order_clerk'@'localhost';
```

RESULT

Grants for order_clerk@localhost

```
GRANT USAGE ON *.* TO `order_clerk`@`localhost`
```

```
GRANT SELECT ON `sales_db`.* TO `order_clerk`@`localhost`
```

```
GRANT INSERT, UPDATE ON `sales_db`.`order_items` TO `order_clerk`@`localhost`
```

```
GRANT INSERT, UPDATE ON `sales_db`.`orders` TO `order_clerk`@`localhost`
```

USAGE ON *.* means only "may log in". The other lines are the grants just made.

Testing the permissions

Reconnect as sales_analyst so the new permissions are picked up. Reading works:

RUN AS SALES_ANALYST

```
USE sales_db;
SELECT COUNT(*) AS orders FROM orders;
```

RESULT

orders

1980

Changing data does not:

RUN AS SALES_ANALYST

```
DELETE FROM customers
WHERE customer_id = 'CG-12520';
```

ERROR

Error Code: 1142. DELETE command denied to user 'sales_analyst'@'localhost' for table 'customers'

Taking a permission away

REVOKE is the reverse of GRANT. Suppose the clerk should capture new orders but no longer change existing ones. Run this as root:

```
REVOKE UPDATE ON sales_db.orders FROM 'order_clerk'@'localhost';
SHOW GRANTS FOR 'order_clerk'@'localhost';
```

RESULT

Grants for order_clerk@localhost

```
GRANT USAGE ON *.* TO `order_clerk`@`localhost`
```

```
GRANT SELECT ON `sales_db`.* TO `order_clerk`@`localhost`
```

```
GRANT INSERT, UPDATE ON `sales_db`.`order_items` TO `order_clerk`@`localhost`
```

```
GRANT INSERT ON `sales_db`.`orders` TO `order_clerk`@`localhost`
```

Why it matters: Give each account the smallest set of permissions that lets it do its job. An analyst who can only read cannot delete a table by accident, and a stolen analyst password cannot be used to change the data.

TRY IT YOURSELF

Create a user called `report_viewer` who can read the `products` table and nothing else. Check the result with `SHOW GRANTS`, then drop the user.

Part 8 Data retrieval

`SELECT` reads data. It never changes anything, so experiment freely. The two required pieces are which columns you want and which table they come from.

Choosing columns

List the columns after `SELECT`, separated by commas. `LIMIT 5` returns only the first five rows, which is useful when you just want to see what a table looks like.

```
SELECT customer_name, segment
FROM customers
LIMIT 5;
```

RESULT

customer_name	segment
Alex Avila	Consumer
Allen Arnold	Consumer
Andrew Allen	Consumer
Adam Bellavance	Home Office
Adrian Barton	Consumer

All columns

An asterisk means every column.

```
SELECT *
FROM products
LIMIT 3;
```

RESULT

product_id	product_name	category	sub_category
FUR-BO-10000112	Bush Birmingham Collection Bookcase, Dark Cherry	Furniture	Bookcases
FUR-BO-10000330	Sauder Camden County Barrister Bookcase, Planked Cherry Finish	Furniture	Bookcases
FUR-BO-10000362	Sauder Inglewood Library Bookcases	Furniture	Bookcases

`SELECT *` is fine for a quick look. In a query you intend to keep, name the columns: the query documents itself and does not change its output if a column is added to the table later.

DISTINCT: removing duplicates

`SELECT segment FROM customers;` returns 730 rows, one per customer, with the same few values repeated. `DISTINCT` returns each value once.

```
SELECT DISTINCT segment
FROM customers;
```

RESULT

segment
Consumer
Home Office
Corporate

With more than one column, `DISTINCT` returns each combination once. This lists every sub-category alongside the category it belongs to:

```
SELECT DISTINCT category, sub_category
FROM products
ORDER BY category, sub_category;
```

RESULT

category	sub_category
Furniture	Bookcases
Furniture	Chairs
Furniture	Furnishings
Furniture	Tables
Office Supplies	Appliances
Office Supplies	Art
Office Supplies	Binders
Office Supplies	Envelopes

First 8 of 17 rows shown.

WHERE: choosing rows

`WHERE` keeps only the rows that pass a test. Text and dates are quoted, numbers are not.

```
SELECT order_id, order_date, city
FROM orders
WHERE state = 'Texas'
LIMIT 5;
```

RESULT

order_id	order_date	city
CA-2014-101602	2014-12-15	El Paso
CA-2014-103849	2014-05-11	Fort Worth
CA-2014-104738	2014-12-30	Laredo
CA-2014-104773	2014-12-08	Houston
CA-2014-105340	2014-11-22	Pasadena

```
SELECT order_id, product_id, sales, profit
FROM order_items
WHERE sales > 8000;
```

RESULT

order_id	product_id	sales	profit
CA-2014-139892	TEC-MA-10000822	8159.9520	-1359.9920
CA-2014-143917	OFF-SU-10000151	8187.6500	327.5060
CA-2017-127180	TEC-CO-10004722	11199.9680	3919.9888
CA-2014-145317	TEC-MA-10002412	22638.4800	-1811.0784

Combine tests with AND. Because order_date is a real date, it can be compared with >=:

```
SELECT order_id, order_date, ship_mode, city
FROM orders
WHERE order_date >= '2017-12-29'
AND ship_mode = 'Standard Class';
```

RESULT

order_id	order_date	ship_mode	city
CA-2017-115427	2017-12-30	Standard Class	Fairfield
CA-2017-118885	2017-12-29	Standard Class	Los Angeles
CA-2017-126221	2017-12-30	Standard Class	Columbus
CA-2017-143259	2017-12-30	Standard Class	New York City

ORDER BY: sorting

Rows come back in no guaranteed order unless you ask for one. ORDER BY sorts ascending; add DESC for descending. Together with LIMIT it answers "top five" questions.

```
SELECT order_id, product_id, sales
FROM order_items
ORDER BY sales DESC
LIMIT 5;
```

RESULT

order_id	product_id	sales
CA-2014-145317	TEC-MA-10002412	22638.4800
CA-2017-127180	TEC-CO-10004722	11199.9680
CA-2014-143917	OFF-SU-10000151	8187.6500
CA-2014-139892	TEC-MA-10000822	8159.9520
US-2017-168116	TEC-MA-10004125	7999.9800

The order of the clauses

The clauses must always be written in this order. Only SELECT and FROM are required.

```
SELECT columns
FROM table_name
WHERE condition
ORDER BY columns
LIMIT number_of_rows;
```

TRY IT YOURSELF

(a) List the distinct regions in `orders`. (b) Show the order ID, order date and city of the five most recent orders. (c) List the name of every product in the sub-category `Copiers`.

Part 9 Using aliases for column names

An alias is a temporary name for a column in the result of one query. It changes the heading you see and nothing in the table.

```
SELECT customer_name AS customer,
       segment       AS customer_type
FROM customers
LIMIT 3;
```

RESULT

customer	customer_type
Alex Avila	Consumer
Allen Arnold	Consumer
Andrew Allen	Consumer

Naming a calculation

Aliases matter most for calculated columns. Without one, the heading is the formula itself. Here `cost` is `sales` minus `profit`:

```
SELECT order_id,
       sales,
       profit,
       sales - profit AS cost
FROM order_items
ORDER BY sales DESC
LIMIT 3;
```

RESULT

order_id	sales	profit	cost
CA-2014-145317	22638.4800	-1811.0784	24449.5584
CA-2017-127180	11199.9680	3919.9888	7279.9792
CA-2014-143917	8187.6500	327.5060	7860.1440

The same applies to functions. DATEDIFF returns the number of days between two dates:

```
SELECT order_id,
       order_date,
       ship_date,
       DATEDIFF(ship_date, order_date) AS days_to_ship
FROM orders
ORDER BY days_to_ship DESC, order_id
LIMIT 3;
```

RESULT

order_id	order_date	ship_date	days_to_ship
CA-2014-100363	2014-04-08	2014-04-15	7
CA-2014-104976	2014-12-09	2014-12-16	7
CA-2014-105893	2014-11-11	2014-11-18	7

- AS is optional (sales - profit cost also works), but writing it makes the query easier to read.
- An alias with a space in it must be wrapped in backticks: AS `Days to ship`. Aliases without spaces are easier to work with.
- An alias can be used in ORDER BY, as above.

An alias cannot be used in WHERE

It is tempting to filter on the new name. It does not work:

```
SELECT order_id,
       DATEDIFF(ship_date, order_date) AS days_to_ship
FROM orders
WHERE days_to_ship = 7;
```

ERROR

Error Code: 1054. Unknown column 'days_to_ship' in 'where clause'

MySQL filters the rows with WHERE before it works out the SELECT list, so at that point days_to_ship does not exist yet. Repeat the calculation in the WHERE:

```
SELECT order_id,
       DATEDIFF(ship_date, order_date) AS days_to_ship
FROM orders
WHERE DATEDIFF(ship_date, order_date) = 7
ORDER BY order_id
LIMIT 3;
```

RESULT

order_id	days_to_ship
CA-2014-100363	7
CA-2014-104976	7
CA-2014-105893	7

Table aliases

A table can have an alias too, usually one or two letters. Columns are then written as `alias.column`. It looks unnecessary with one table and becomes essential when a query uses several, as in Part 12.

```
SELECT o.order_id, o.order_date, o.city
FROM orders AS o
WHERE o.region = 'South'
ORDER BY o.order_date
LIMIT 3;
```

RESULT

order_id	order_date	city
CA-2014-149020	2014-01-10	Springfield
CA-2014-109232	2014-01-13	Mount Pleasant
CA-2014-162775	2014-01-13	Bossier City

TRY IT YOURSELF

Show each order item's `order_id`, its sales, its quantity and the price per unit (sales divided by quantity) with the heading `unit_price`. List the five highest unit prices.

Part 10 Comments in SQL

A comment is text that MySQL ignores. It is there for the next person who reads the query, who is often you in three months.

```
-- A single-line comment starts with two dashes and a space
SELECT customer_name, segment      -- a comment can follow code on the same line
FROM customers
LIMIT 3;
```

```
/* A multi-line comment starts with slash-star
   and carries on over as many lines as needed
   until star-slash. */
SELECT customer_name, segment
FROM customers
LIMIT 3;
```

Watch out: In MySQL the two dashes must be followed by a space. `--comment` is not a comment and produces an error. MySQL also accepts `#` to start a single-line comment, but other database systems do not, so prefer the two dashes.

Switching a line off while testing

Commenting out part of a query is the quickest way to test it with and without a condition. Here the second condition is switched off without being deleted:

```
SELECT order_id, order_date, city, state
FROM orders
WHERE region = 'West'
-- AND state = 'California'
ORDER BY order_date
LIMIT 3;
```

RESULT

order_id	order_date	city	state
US-2014-117163	2014-01-27	San Diego	California
CA-2014-115161	2014-01-31	Mission Viejo	California
CA-2014-139857	2014-02-02	San Diego	California

Documenting a saved script

A block comment at the top of a script says what it is for. Inside the script, use comments to explain why something is done, since the SQL already says what is done.

```
/*
  Script : top_order_lines.sql
  Purpose: the five largest order lines, for the weekly sales review
*/
SELECT order_id, product_id, sales
FROM order_items
-- Full-price lines only: discounted lines are reviewed separately
WHERE discount = 0
ORDER BY sales DESC
LIMIT 5;
```

Part 11 Temporary tables

A temporary table is a real table that exists only for your current connection. Only you can see it, and MySQL drops it automatically when you disconnect. Use one to hold an intermediate result you want to query several times, without touching the permanent tables.

Creating one from a query

The quickest way is `CREATE TEMPORARY TABLE ... AS SELECT`. The columns and their types are copied from the query result.

```
CREATE TEMPORARY TABLE tmp_west_orders AS
SELECT order_id, order_date, customer_id, city, state
FROM orders
WHERE region = 'West';
```

OUTPUT 609 row(s) affected

From here on it behaves like any other table:

```
SELECT COUNT(*) AS west_orders
FROM tmp_west_orders;
```

RESULT

west_orders
609

```
SELECT DISTINCT state
FROM tmp_west_orders
ORDER BY state
LIMIT 5;
```

RESULT

state
Arizona
California
Colorado
Idaho
Montana

Defining the columns first

You can also define the columns yourself, exactly as in `CREATE TABLE`, and fill the table with `INSERT`.

```
CREATE TEMPORARY TABLE tmp_big_lines (
  order_id  VARCHAR(20),
  product_id VARCHAR(20),
  sales     DECIMAL(10,4)
);

INSERT INTO tmp_big_lines (order_id, product_id, sales)
SELECT order_id, product_id, sales
FROM order_items
WHERE sales > 8000;

SELECT * FROM tmp_big_lines
ORDER BY sales DESC;
```

RESULT

order_id	product_id	sales
CA-2014-145317	TEC-MA-10002412	22638.4800
CA-2017-127180	TEC-CO-10004722	11199.9680
CA-2014-143917	OFF-SU-10000151	8187.6500
CA-2014-139892	TEC-MA-10000822	8159.9520

How temporary tables behave

Temporary tables do not appear in `SHOW TABLES`. Both of ours exist right now, yet only the five permanent tables are listed:

```
SHOW TABLES;
```

RESULT

Tables_in_sales_db
customers
order_items
orders
products
sales_raw

- They belong to your connection. In Workbench, closing the connection tab ends the connection and the temporary tables go with it. Another user can have a temporary table with the same name without any clash.
- A temporary table cannot be used twice in the same query. Doing so gives `Error Code: 1137. Can't reopen table.`

To remove one before you disconnect, drop it:

```
DROP TEMPORARY TABLE tmp_big_lines;
```

Tip: Creating a temporary table needs its own permission. The read-only analyst from Part 7 cannot do it until root runs: `GRANT CREATE TEMPORARY TABLES ON sales_db.* TO 'sales_analyst'@'localhost';`

Part 12 CTEs (Common Table Expressions)

A CTE is a named result that exists only for the one query it is written in. It starts with `WITH`, gives a query a name, and the main query underneath then uses that name as if it were a table.

```
WITH cte_name AS (
    -- a SELECT that builds the intermediate result
)
SELECT ...
FROM cte_name;
```

A first CTE

This is the West region example from Part 11, written as a CTE. The whole thing is one statement, with one semicolon at the very end.

```
WITH west_orders AS (  
    SELECT order_id, order_date, customer_id, city, state  
    FROM orders  
    WHERE region = 'West'  
)  
SELECT order_id, order_date, city  
FROM west_orders  
ORDER BY order_date DESC, order_id  
LIMIT 3;
```

RESULT

order_id	order_date	city
CA-2017-115427	2017-12-30	Fairfield
CA-2017-118885	2017-12-29	Los Angeles
CA-2017-129805	2017-12-28	Seattle

Once that statement has finished, `west_orders` is gone. Using it in a separate statement fails:

```
SELECT * FROM west_orders;
```

ERROR

```
Error Code: 1146. Table 'sales_db.west_orders' doesn't exist
```

A CTE that prepares a calculation

CTEs earn their place when a query has to be built in steps. Question: which five orders have the highest total sales, and who placed them? An order's total is not stored anywhere. It has to be added up from the order's items first, and then matched to the order and the customer.

```
WITH order_totals AS (
  SELECT order_id,
         SUM(sales) AS order_sales,
         SUM(profit) AS order_profit
  FROM order_items
  GROUP BY order_id
)
SELECT o.order_id,
       o.order_date,
       c.customer_name,
       t.order_sales,
       t.order_profit
FROM order_totals AS t
JOIN orders AS o ON o.order_id = t.order_id
JOIN customers AS c ON c.customer_id = o.customer_id
ORDER BY t.order_sales DESC
LIMIT 5;
```

RESULT

order_id	order_date	customer_name	order_sales	order_profit
CA-2014-145317	2014-03-18	Sean Miller	23661.2280	-1789.7327
CA-2017-127180	2017-10-22	Tom Ashbrook	13716.4580	4597.1657
CA-2014-139892	2014-09-08	Becky Martin	10539.8960	-1878.7892
CA-2014-143917	2014-07-25	Ken Lonsdale	8319.2900	376.8324
US-2017-168116	2017-11-04	Grant Thornton	8167.4200	-3825.3394

Read it in two steps:

- The CTE.** `GROUP BY order_id` collapses the 4,000 order items into one row per order, and `SUM` adds up the sales and profit of each order's items. The result is named `order_totals`.
- The main query.** It starts from `order_totals` and uses `JOIN` to look up the matching row in `orders` (for the date) and then in `customers` (for the name). The foreign keys from Part 4 are what make the matches possible. The table aliases `t`, `o` and `c` say which table each column comes from.

`GROUP BY` and `JOIN` are Day 2 topics. What matters today is the shape: name a step, then use it.

More than one CTE

Separate CTEs with a comma. A later CTE can use an earlier one, so a long query reads from top to bottom as a series of small steps.

```
WITH order_totals AS (
  SELECT order_id, SUM(sales) AS order_sales
  FROM order_items
  GROUP BY order_id
),
big_orders AS (
  SELECT order_id, order_sales
  FROM order_totals
  WHERE order_sales > 5000
)
SELECT COUNT(*) AS big_order_count
FROM big_orders;
```

RESULT

big_order_count
8

Temporary table or CTE?

	Temporary table	CTE
Lives for	The whole connection	One statement
Created with	CREATE TEMPORARY TABLE	WITH name AS (...)
Result is kept	Yes, until you drop it or disconnect	No, it is worked out each time the query runs
Use it when	Several separate queries need the same intermediate result	One query needs to be broken into readable steps
Permission	CREATE TEMPORARY TABLES	Only SELECT

Watch out: CTEs need MySQL 8.0 or later. On MySQL 5.7, WITH is a syntax error.

TRY IT YOURSELF

(a) Create a temporary table called `tmp_tech_products` holding the ID and name of every product in the Technology category, and count its rows. (b) Write a CTE called `big_lines` containing the order items with sales over 5000, then select everything from it, largest sale first.

Appendix A: Common errors and what they mean

Workbench shows errors in the Output panel as **Error Code:** followed by a number. The number is the quickest thing to look up.

Code	Message begins	What it means and what to do
1007	Can't create database; database exists	It was already created. Carry on, or use <code>IF NOT EXISTS</code> .
1046	No database selected	Run <code>USE sales_db;</code> first.
1050	Table already exists	It was already created. Drop it first if you need to change it.
1054	Unknown column	A misspelt column name, an alias used in <code>WHERE</code> , or a text value written without quotes.
1062	Duplicate entry for key PRIMARY	A row with that primary key value is already in the table.
1064	You have an error in your SQL syntax	A typing mistake: a missing comma, quote or bracket, or a misspelt keyword. Look just before the text quoted in the message.
1136	Column count doesn't match value count	The number of values is not the same as the number of columns listed.
1142	command denied to user	The account you are logged in as lacks that permission.
1146	Table doesn't exist	A misspelt table name, the wrong database, or a temporary table or CTE that no longer exists.
1175	You are using safe update mode	An <code>UPDATE</code> or <code>DELETE</code> whose <code>WHERE</code> does not use a key column. Filter on the primary key.
1292	Incorrect date value	The date is not written year-month-day.
1406	Data too long for column	The text is longer than the column's <code>VARCHAR(n)</code> limit.
1451	Cannot delete or update a parent row	Other rows still refer to this one. Delete those first.
1452	Cannot add or update a child row	The foreign key value does not exist in the parent table.
3948, 2068	Loading local data is disabled / file request rejected	<code>LOAD DATA LOCAL</code> is not enabled on both the server and the client. See Part 5.

Appendix B: Answers to the exercises

Part 1

```
CREATE DATABASE practice_db;
SHOW DATABASES;
DROP DATABASE practice_db;
USE sales_db;
```

Part 2

```
CREATE TABLE regions (
    region_name VARCHAR(20),
    manager     VARCHAR(50)
);
DESCRIBE regions;
DROP TABLE regions;
```

Part 3

- (a) VARCHAR(15). A cellphone number starts with a zero or a plus sign, and nobody adds phone numbers together.
- (b) DECIMAL(6,2). An exact value with decimals, up to 9999.99 kg.
- (c) DATE.
- (d) INT. A whole number.

Part 4

```
CREATE TABLE returns (
    return_id      INT          AUTO_INCREMENT PRIMARY KEY,
    order_item_id  INT          NOT NULL,
    return_date    DATE         NOT NULL,
    reason         VARCHAR(255),
    FOREIGN KEY (order_item_id) REFERENCES order_items (order_item_id)
);
```

A return is a fact about one order item, so the foreign key points at `order_items`. One order item can have several returns (two of five chairs today, one more next week), so `returns` gets its own `AUTO_INCREMENT` key. Run `DROP TABLE returns;` afterwards to keep the database as designed.

Part 5

```
INSERT INTO customers (customer_id, customer_name, segment)
VALUES ('ZZ-00001', 'Your Name', 'Consumer');

SELECT * FROM customers
WHERE customer_id = 'ZZ-00001';

DELETE FROM customers
WHERE customer_id = 'ZZ-00001';
```

Part 7

```
CREATE USER 'report_viewer'@'localhost' IDENTIFIED BY 'Viewer#2026';
GRANT SELECT ON sales_db.products TO 'report_viewer'@'localhost';
SHOW GRANTS FOR 'report_viewer'@'localhost';
DROP USER 'report_viewer'@'localhost';
```

Part 8

```
-- (a)
SELECT DISTINCT region
FROM orders;

-- (b)
SELECT order_id, order_date, city
FROM orders
ORDER BY order_date DESC
LIMIT 5;

-- (c)
SELECT product_name
FROM products
WHERE sub_category = 'Copiers';
```

Part 9

```
SELECT order_id,
       sales,
       quantity,
       sales / quantity AS unit_price
FROM order_items
ORDER BY unit_price DESC
LIMIT 5;
```

Parts 11 and 12

```
-- (a)
CREATE TEMPORARY TABLE tmp_tech_products AS
SELECT product_id, product_name
FROM products
WHERE category = 'Technology';

SELECT COUNT(*) AS tech_products
FROM tmp_tech_products;

-- (b)
WITH big_lines AS (
  SELECT order_id, product_id, sales
  FROM order_items
  WHERE sales > 5000
)
SELECT *
FROM big_lines
ORDER BY sales DESC;
```

Starting again from scratch

To wipe everything created in this workbook and begin again, run these as root:

```
DROP DATABASE IF EXISTS sales_db;
DROP USER IF EXISTS 'sales_analyst'@'localhost';
DROP USER IF EXISTS 'order_clerk'@'localhost';
DROP USER IF EXISTS 'sales_admin'@'localhost';
```